

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 3

Module:3	String Matching Algorithms
----------	----------------------------

5 hours

Naïve String-matching Algorithms,
KMP algorithm,
Rabin-Karp Algorithm,
Suffix Trees.

Suffix Trees (Introduction)

- Suffix trees are data structures used in string matching and manipulation.
- A suffix tree is built from the given text.
- After preprocessing the text (building the suffix tree of the text), we can search for any pattern in $O(m)$ time, where m is the length of the pattern.
- Here, all the suffixes of a given string in a way that enables efficient pattern searching, substring retrieval, and other operations.

What is Suffix?

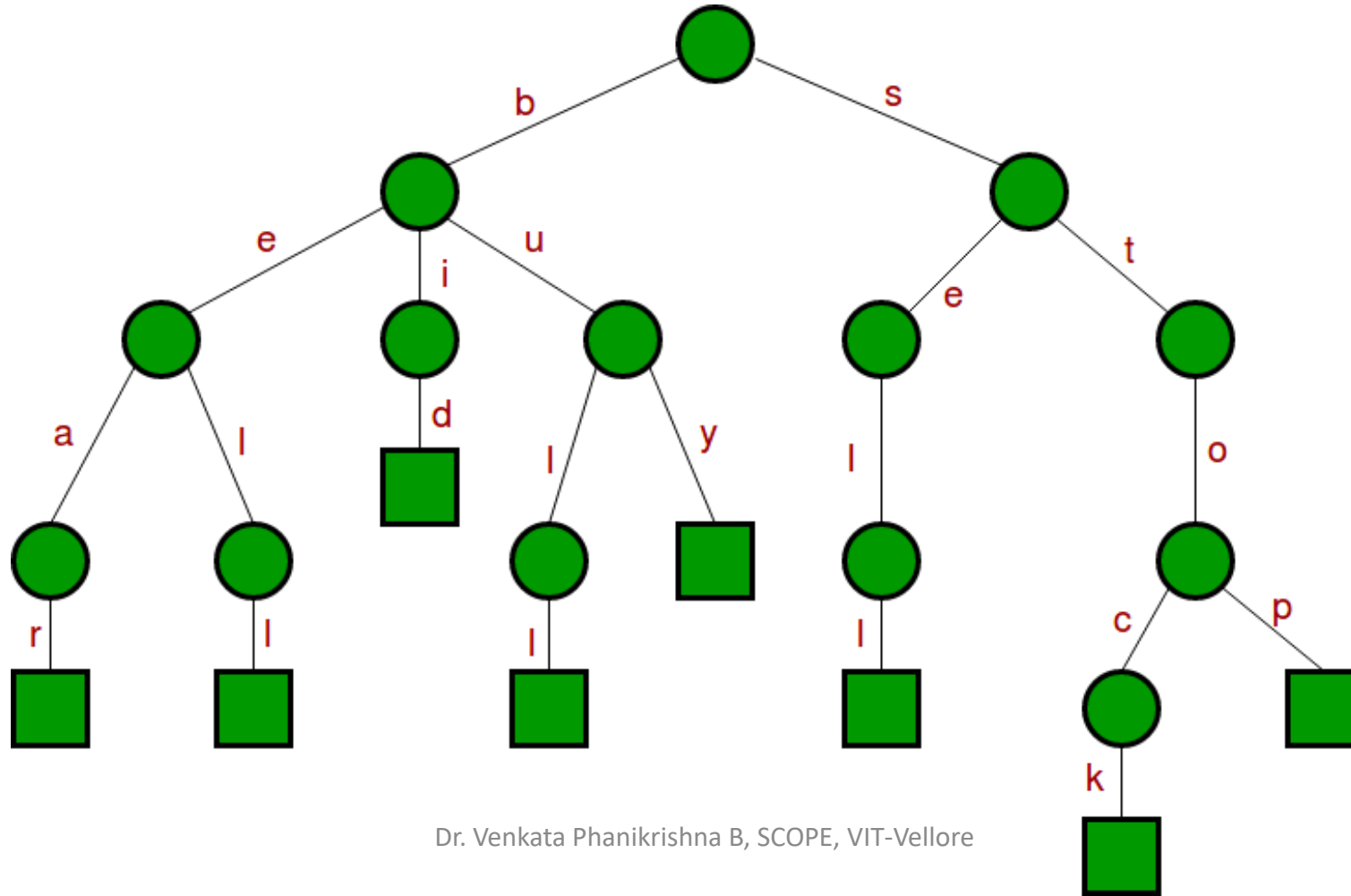
Suffix means a letter or group of letters that you add at the end of a word

Suffix Trees (Introduction)

A suffix tree for a given text is a compressed trie for all suffixes of the text.

Compressed Trie with the following array of words:

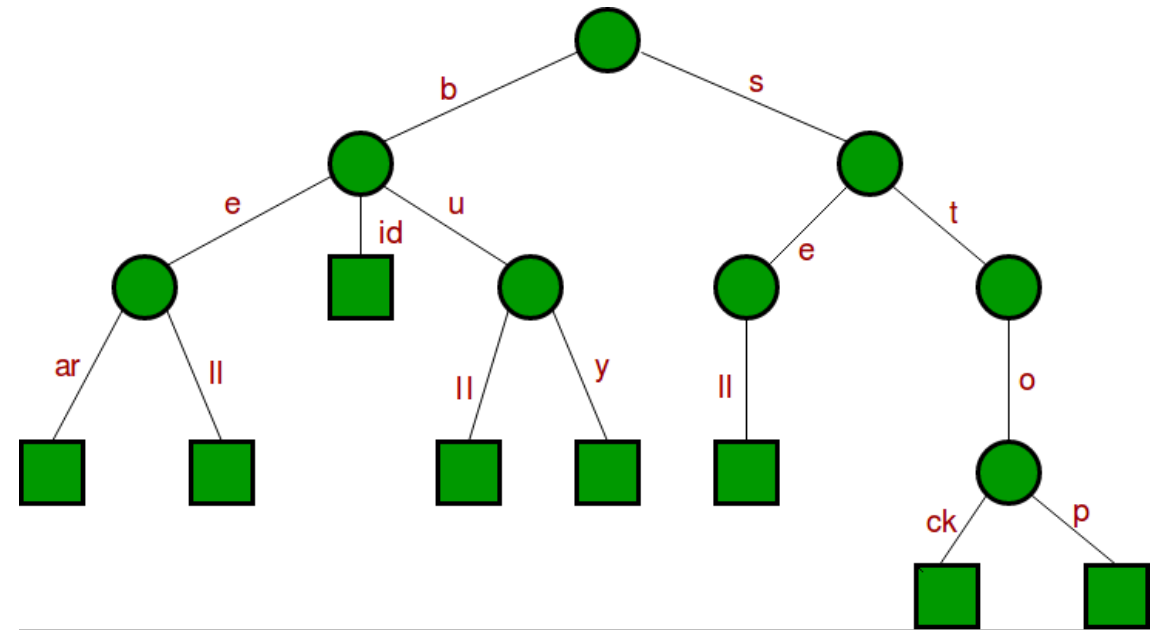
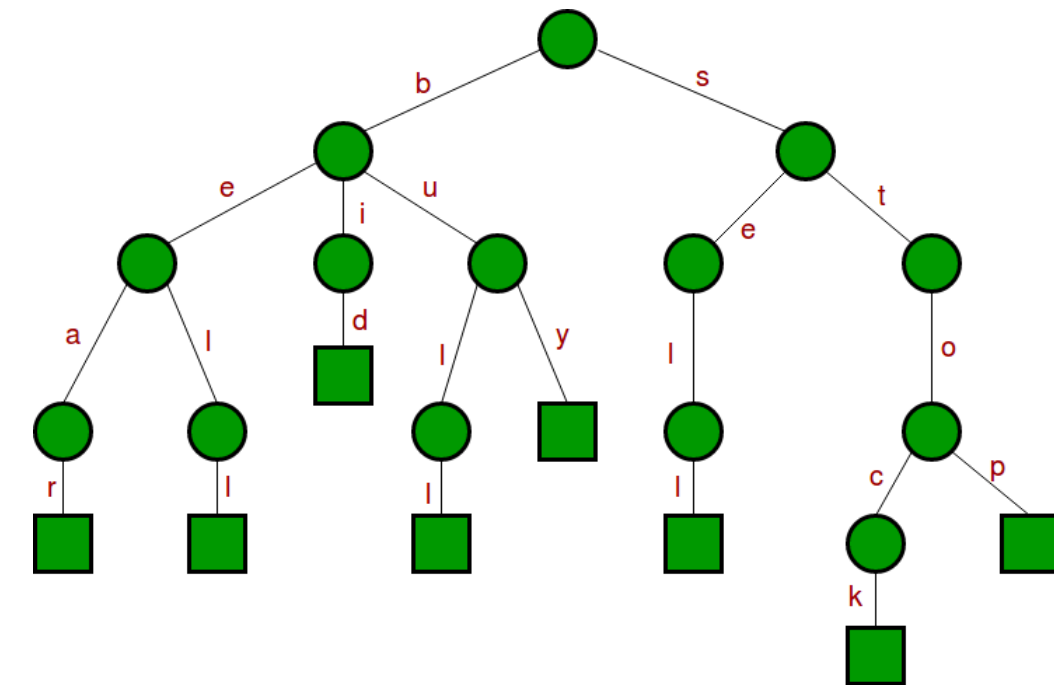
{bear, bell, bid, bull, buy, sell, stock, stop}



Suffix Trees (Introduction)

Compressed Trie with the following array of words: {bear, bell, bid, bull, buy, sell, stock, stop}

Compress Trie is obtained from standard trie by joining chains of single nodes. The nodes of a compressed trie can be stored by storing index ranges at the nodes.



Build a Suffix Tree for a given text

- Suffix Tree is compressed trie of all suffixes,
 1. Generate all suffixes of given text.
 2. Consider all suffixes as individual words and build a compressed trie.

Example: “banana”

“banana\0”,

where ‘\0’ is string termination character.

Build a Suffix Tree for a given text

- Suffix Tree is compressed trie of all suffixes,
 1. **Generate all suffixes of given text.**
 2. Consider all suffixes as individual words and build a compressed trie.

Example: “banana”
“banana\0”,
where ‘\0’ is string termination character.

Following are all suffixes of “banana\0”
banana\0
anana\0
nana\0
ana\0
na\0
a\0
\0

Build a Suffix Tree for a given text

- Suffix Tree is compressed trie of all suffixes,
 1. Generate all suffixes of given text.
 2. Consider all suffixes as individual words and build a compressed trie.

Example: “banana”

“banana\0”,

where ‘\0’ is string termination character.

Following are all suffixes of “banana\0”

banana\0

anana\0

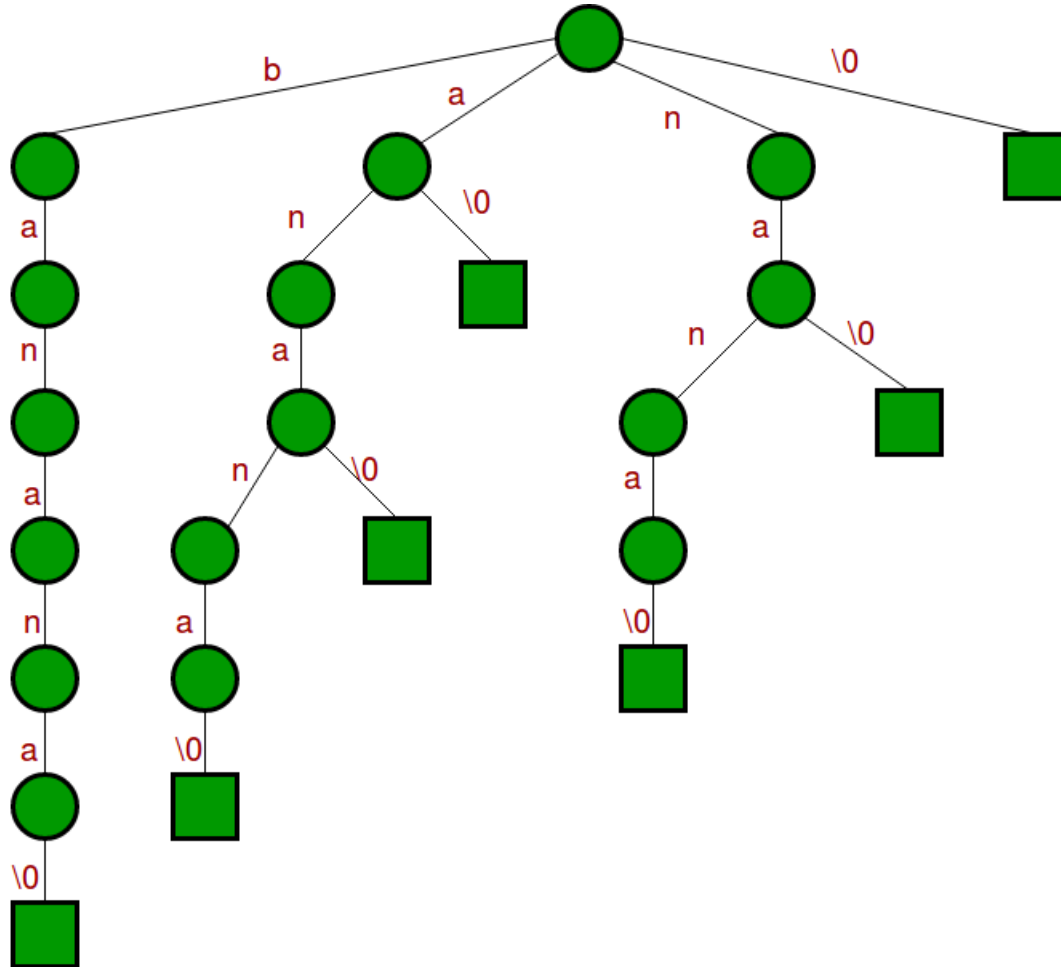
nana\0

ana\0

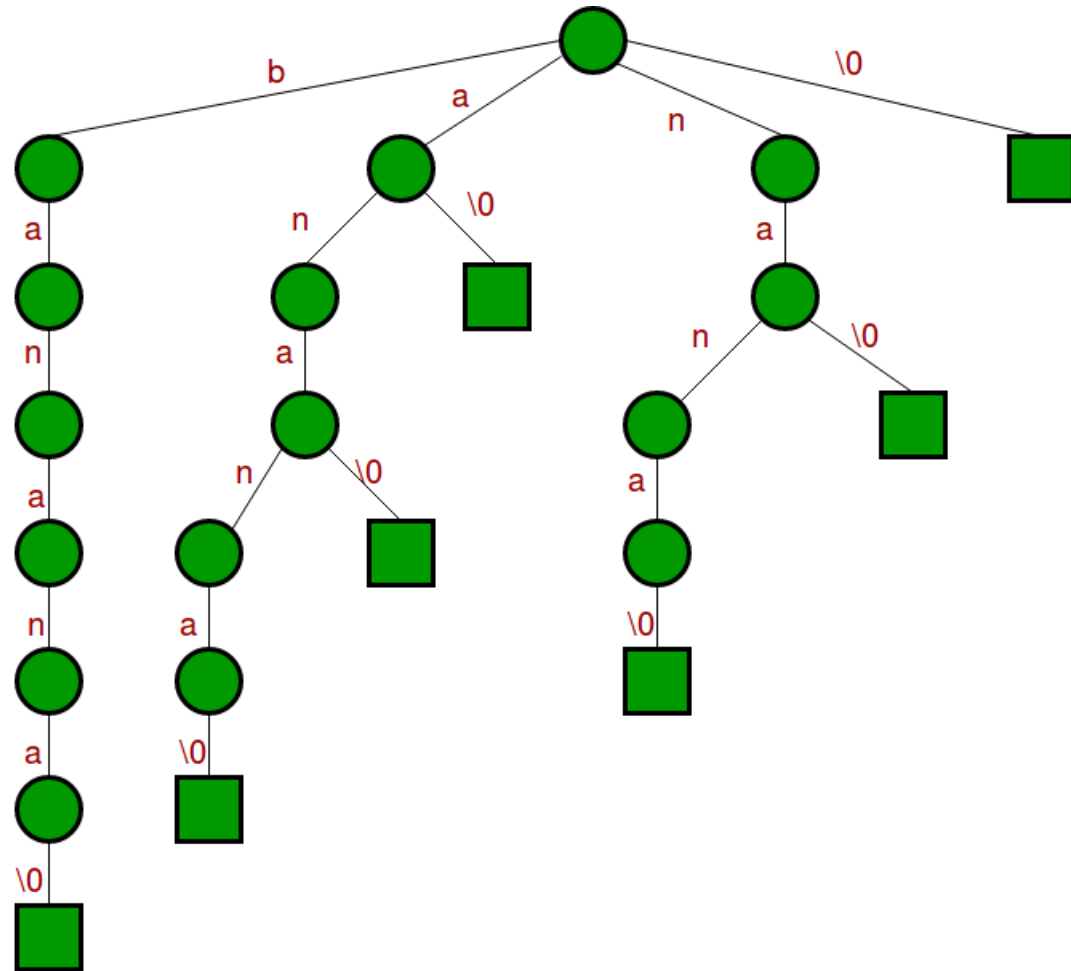
na\0

$$a \setminus 0$$

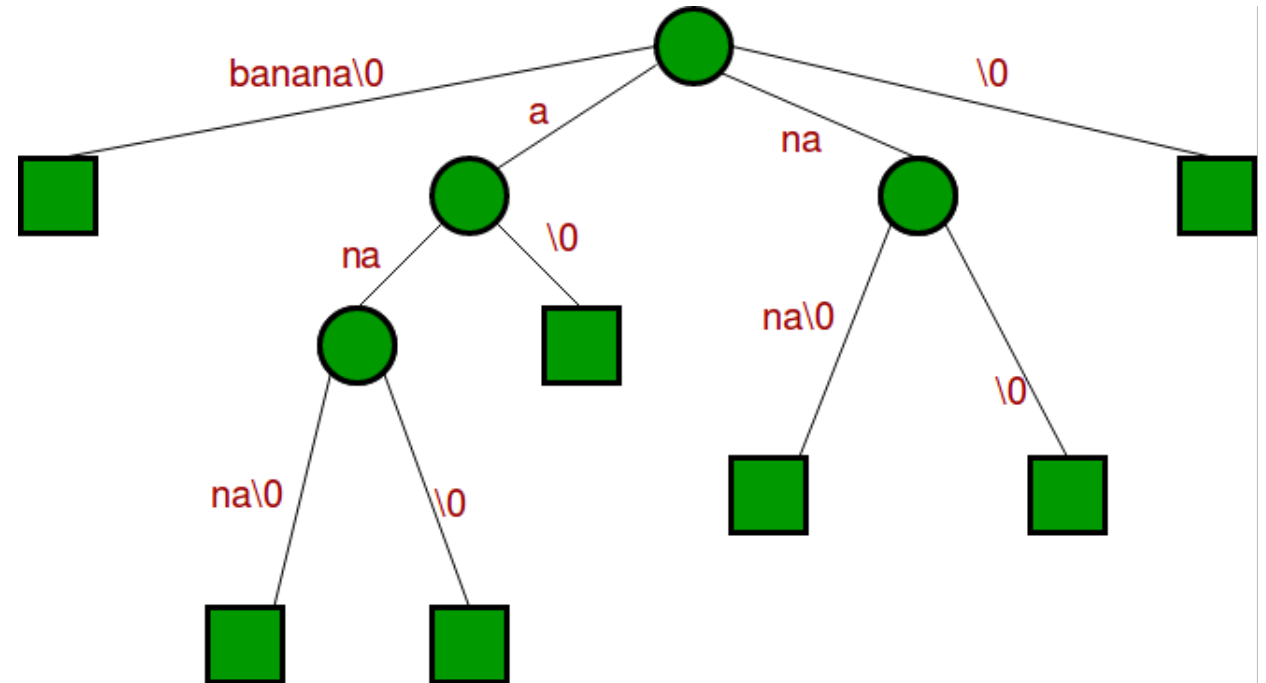
\0



Build a Suffix Tree for a given text



If we join chains of single nodes, we get the following compressed trie, which is the Suffix Tree for given text "banana\0"



Searching

Steps to search a pattern in the built Suffix Tree:

1. Starting from the first character of the pattern and the root of the Suffix Tree, do the following for every character:
 - a) For the current character of the pattern, if there is an edge from the current node of the suffix tree, follow the edge.
 - b) If there is no edge, print "Pattern doesn't exist in the text" and return.
2. If all characters of the pattern have been processed (i.e., there is a path from the root for the characters of the given pattern), then print "Pattern found".

Searching

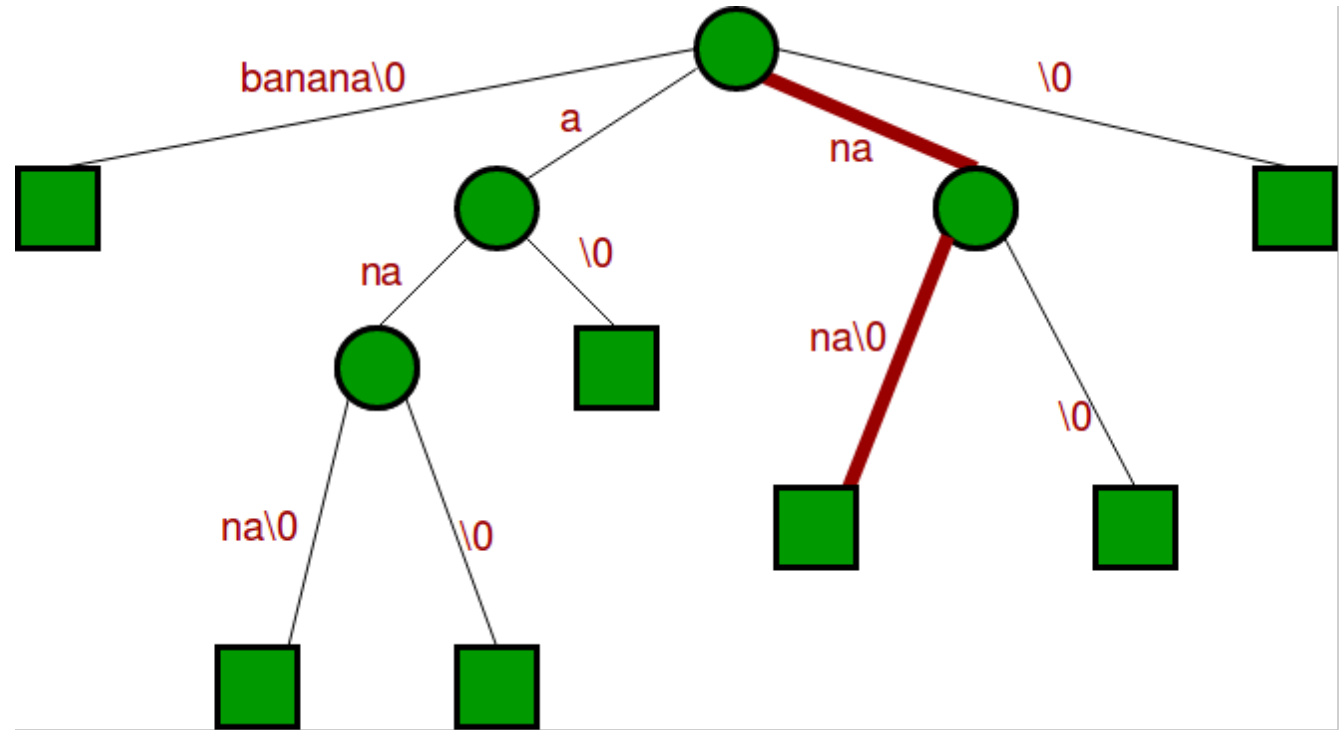
Steps to search a pattern in the built Suffix Tree:

1. Starting from the first character of the pattern and the root of the Suffix Tree, do the following for every character:

- For the current character of the pattern, if there is an edge from the current node of the suffix tree, follow the edge.
- If there is no edge, print "Pattern doesn't exist in the text" and return.

2. If all characters of the pattern have been processed (i.e., there is a path from the root for the characters of the given pattern), then print "Pattern found".

Let us consider the example pattern as “nan” to see the searching process. Following diagram shows the path followed for searching “**nan**” or “**nana**”.



Time Complexity

- Constructing the suffix tree: $O(n^2)$
- Searching for a pattern: $O(m)$, where m is the length of the pattern.
- This is called a generalized suffix tree

Applications of Suffix Trees:

1. Pattern Searching.
2. Finding the longest repeated substring.
3. Finding the longest common substring.
4. Finding the longest palindrome in a string.

Method	Procedure	Time complexity (Matching Time)	Preprocessing	Best in situation	Other observations
Brute Force	Direct matching	$O(m*n)$	NA	If text and pattern are very small	$(m-n+1)$ index values are compared with pattern
KMP	LPS	$O(M+N)$	$O(M)$	Overlapping and non overlapping	
Rabin	Hash function	$O(m*n)$ $O(M+N)$	$O(M)$	Best for repeated patterns	Two parameters, d and prime number.
Suffix Tree	Suffix tree	$O(M)$	$O(N^2)$	Longest Repeated Patterns	

Any

Question



PresenterMedia



Thank You!

**FOR YOUR
ATTENTION**

